

passgen

Детерминированный генератор паролей

*Полная документация для защиты проекта
Подробное описание каждой функции программы*

Сгенерировано 8 июля 2026 г.

1. Идея проекта и решаемая проблема

В современном мире у каждого человека есть множество аккаунтов: электронная почта, социальные сети, интернет-банкинг, рабочие системы, учебные порталы, форумы, стриминговые сервисы и десятки других. Для каждого требуется пароль. Использовать один и тот же пароль везде очень опасно: если злоумышленник узнает пароль от одного сервиса (например, из-за утечки базы данных), он получит доступ ко всем аккаунтам. Запоминать 20-30 разных сложных паролей человеческая память не позволяет.

Менеджеры паролей (LastPass, Bitwarden, 1Password) хранят все пароли в одном зашифрованном месте, но это создает единую точку отказа. Если злоумышленник получит доступ к мастер-паролю от менеджера, все пароли окажутся скомпрометированы. Кроме того, менеджеры требуют установки дополнительного программного обеспечения, синхронизации между устройствами и доверия к облачному хранилищу.

passgen предлагает принципиально другой подход — детерминированную генерацию. Программа не хранит пароли вообще. Вместо этого она каждый раз вычисляет пароль заново с помощью криптографического алгоритма HMAC-SHA256. Достаточно запомнить одну секретную фразу (сид) и знать, для какого сервиса нужен пароль. Программа всегда выдаст один и тот же пароль для одинаковых входных данных. Это означает, что пароль можно восстановить на любом устройстве, в любой момент времени, без необходимости носить с собой базу данных паролей или синхронизировать её через облако.

Преимущества такого подхода: нет единой точки отказа, нет необходимости в облачном хранилище, нет риска кражи базы паролей. Даже если устройство будет полностью уничтожено, все пароли останутся доступны — нужно лишь вспомнить сид. При этом для каждого сервиса создаётся уникальный, криптостойкий пароль, который невозможно подобрать или предсказать.

2. Структура и архитектура программы

Программа состоит из одного HTML-файла (index.html), который содержит всё необходимое: разметку (HTML), стили (CSS) и логику (JavaScript). Для работы программы не требуется устанавливать дополнительные библиотеки, пакеты или программы. Достаточно любого современного браузера с поддержкой Web Crypto API (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera). Программа полностью работает на стороне клиента, никакие данные не передаются по сети и не сохраняются на сервере.

2.1. HTML — разметка страницы

HTML-код определяет структуру интерфейса. Основные элементы: поле ввода сида (секретной фразы), поле ввода названия сервиса, панель с популярными сервисами, выпадающий список пресетов, регулятор длины пароля, переключатели типов символов (заглавные, строчные, цифры, спецсимволы), расширенные настройки (первая заглавная, последняя цифра), кнопка генерации, поле вывода логина и пароля, индикатор надёжности, кнопка копирования, ссылки экспорта и импорта, модальное окно импорта.

2.2. CSS — стили и оформление

Оформление выполнено в современном минималистичном стиле с использованием CSS-переменных. Цветовая схема: светло-фиолетовый фон, белые карточки с тенью, тёмный текст, фиолетовые акценты. Кнопки имеют градиентную заливку, скруглённые углы и тени. Реализованы анимации: fadeUp для появления элементов, glow для пульсации ошибки, slideDown/slideUp для раскрытия панелей, checkPop для переключателей. Переключатели символов выполнены в виде кружочков (кастомные чекбоксы).

2.3. JavaScript — логика программы

Вся логика написана на JavaScript с использованием Web Crypto API — встроенной в браузер криптографической библиотеки. Все криптографические операции (хеширование, HMAC) выполняются нативным кодом браузера. Программа содержит 26 функций, каждая из которых отвечает за определённую задачу.

3. Криптографический алгоритм

3.1. Что такое HMAC-SHA256 и чем отличается от обычного хеша

SHA-256 (Secure Hash Algorithm 256 бит) — это криптографическая хеш-функция, которая принимает входные данные произвольной длины и возвращает фиксированный 256-битный (32-байтный) дайджест. Хеш-функция обладает свойствами: необратимость (по хешу нельзя восстановить исходные данные), детерминированность (одинаковые входные данные всегда дают одинаковый хеш), лавинный эффект (изменение одного бита на входе меняет половину битов на выходе), устойчивость к коллизиям (сложно найти два разных сообщения с одинаковым хешем).

HMAC (Hash-based Message Authentication Code) — конструкция, использующая хеш-функцию вместе с секретным ключом. Аналогия: SHA-256 — это чёрный ящик, который превращает любые данные в 32 байта. Любой может вычислить SHA-256 от любого сообщения. HMAC — это сейф с ключом: чтобы вычислить HMAC, нужно знать секретный ключ. Даже если алгоритм HMAC публичный, без знания ключа невозможно вычислить правильный HMAC.

В passgen HMAC-SHA256 используется так: секретная фраза пользователя (сид) становится ключом HMAC, а сообщением — строка из названия сервиса и длины пароля. Это гарантирует: (1) без сида нельзя вычислить пароль; (2) для разных сервисов — разные пароли; (3) для одного сервиса пароль всегда одинаков (детерминированность); (4) изменение сида полностью меняет все пароли.

3.2. Почему HMAC, а не просто SHA-256 от seed + service

Если бы программа использовала просто SHA-256(seed + service), это было бы уязвимо к атаке добавления (length extension attack). SHA-256 уязвим: зная SHA-256(M), можно вычислить SHA-256(M + padding + extra) без знания M. HMAC не подвержен этой атаке, так как использует ключ дважды: XOR с внутренним паддингом, хеширование с сообщением, затем хеширование результата с ключом и внешним паддингом.

3.3. Rejection Sampling — обеспечение равномерного распределения

При превращении байтов HMAC в символы пароля возникает математическая проблема. Каждый байт может быть от 0 до 255 (256 значений). Длина алфавита может быть 50 или 62. 256 не делится нацело ни на 50, ни на 62.

Если бы программа просто брала байт % длина_алфавита, некоторые символы встречались бы чаще других. Пример: алфавит из 50 символов. Байты 0-249 дают каждый символ по 5 раз, но байты 250-255 (6 штук) дают символы 0-5 дополнительно. Это создаёт смещение (bias), недопустимое в криптографии.

Решение: $\text{max} = 256 - (256 \% \text{длина_алфавита})$. Байты $\geq \text{max}$ отбрасываются (reject). Используются только байты $< \text{max}$. Поскольку max кратен длине алфавита, распределение символов строго равномерное. Пример: алфавит 50, $\text{max} = 250$, байты 250-255 отбрасываются, каждый символ встречается ровно 5 раз. Вероятность каждого символа = $1/50$.

Этот метод называется Rejection Sampling — стандартная криптографическая техника. Недостаток: теоретически может потребоваться больше одной итерации HMAC, если все 32 байта отбракованы. На практике вероятность отбраковки одного байта = $6/256 = 2.3\%$, вероятность отбраковки всех 32 байтов ничтожно мала.

4. Пользовательский интерфейс

Интерфейс программы состоит из следующих элементов, расположенных сверху вниз:

4.1. Заголовок и кнопки экспорта/импорта

Заголовок 'passgen' и ссылки 'Экспорт' (выгрузка настроек в JSON) и 'Импорт' (загрузка из JSON). При нажатии на 'Импорт' открывается системное окно выбора файла.

4.2. Карточка 'Доступ' (сид)

Поле ввода сида — секретной фразы, которую пользователь держит в голове. Под полем отображается пояснение 'Секретная фраза, которую вы держите в голове.' При пустом сиде поле подсвечивается розовой обводкой с анимацией пульсации, появляется сообщение 'Введите сид'.

4.3. Карточка 'Сервис'

Поле ввода названия сервиса (google, github, vk). Справа — ссылка 'список' с панелью популярных сервисов. Каждый сервис — чип с логотипом. При клике название подставляется. Пользовательские сервисы добавляются автоматически при генерации, отображаются без логотипа. Удаление сохранённых сервисов — по ПКМ.

4.4. Карточка 'Параметры'

Пресеты (Свой, Минимальный, Стандартный, Максимальный), регулятор длины (1-99) с кнопками +/- , переключатели символов (кружочки), расширенные настройки (Первая заглавная, Последняя цифра) под сворачиваемой панелью.

4.5. Кнопка 'Создать'

Запускает gen(): проверка ввода, HMAC-генерация, инверсии, вывод, автоматическое копирование в буфер.

4.6. Карточка 'Логин и пароль'

Поле вывода логина с кнопкой генерации, поле пароля (только чтение), индикатор надежности (три полосы + текст с эмодзи).

4.7. Кнопка 'Копировать'

Копирует пароль в буфер. 'Скопировано!' на 1.5 сек.

4.8. Модальное окно импорта

При импорте JSON с несколькими сервисами — выбор конкретных сервисов. По умолчанию все отмечены.

5. Описание каждой функции программы

В этом разделе представлено подробное описание всех 26 функций. Каждая рассмотрена с точки зрения назначения, принципа работы, входных и выходных данных, особенностей.

5.1. Функция `gen()` — генерация пароля

Назначение: главная функция, запускается при нажатии 'Создать' или Enter в полях. Порядок работы: (1) чтение `seed` из поля — если пусто, розовая подсветка, сообщение 'Введите сид', `return`. (2) чтение `svc` (если пусто — '(empty)'), `len` (если некорректно — 10). (3) проверка `state` — если ни один тип символов не выбран, сообщение 'Выберите хотя бы один тип'. (4) формирование `alpha` из СНК по включённым типам. (5) вызов `hmacGen(seed, svc, len, alpha)` — `await`. (6) применение инверсий: `inv.cap` → `toUpperCase()` первого символа; `inv.dig` → отдельный HMAC (`seed+'_lastdig'` как ключ, `svc+len` как сообщение, `байт[0] % 10` — цифра). (7) вывод пароля в `#pwd`, `updateStrength()`. (8) автокопирование в буфер. (9) если `svc` новый — добавление в `svcs`, `renderSvcChips()`. (10) `saveCfg()`.

Функция асинхронная (`async`), так как `hmacGen()` использует `await` для криптоопераций.

5.2. Функция hmacGen() — криптографическая генерация пароля

Назначение: ядро программы — детерминированная генерация на основе HMAC-SHA256 с Rejection Sampling. Параметры: seed (ключ HMAC), svc (часть сообщения), len (длина пароля), alpha (алфавит символов). Возвращает строку пароля.

Алгоритм работы:

- (1) Подготовка ключа: seed кодируется в байты TextEncoder(), создаётся ключ `crypto.subtle.importKey('raw', keyBytes, {name:'HMAC', hash:'SHA-256'}, false, ['sign'])`. false — ключ нельзя экспортировать.
- (2) Подготовка сообщения: svc+len кодируется в байты — база сообщения.
- (3) $\text{max} = 256 - (256 \% \text{alpha.length})$ — для Rejection Sampling.
- (4) Цикл while (`pwd.length < len`): к msg добавляется байт счётчика ct, вычисляется HMAC от msg, 32 байта перебираются: если байт < max — символ = `alpha[byte \% alpha.length]` добавляется к pwd.
- (5) Если 32 байта не хватило — `ct++` и новая итерация.

Криптостойкость: детерминированность, равномерность (Rejection Sampling), необратимость, лавинный эффект, стойкость к коллизиям.

5.3. Функция genLogin() — генерация логина

Назначение: создание простого запоминающегося логина на основе сида и сервиса. Порядок: (1) очистка поля. (2) проверка seed (пусто → ошибка). (3) проверка svc (пусто → return). (4) `loginCounter++`. (5) `SHA-256(seed+'login'+svc+loginCounter)`. (6) первые 6 байт → alpha 'abcdefghijklmnopqrstuvwxyz23456789' (без i,l,o,0,1 — чтоб не путать). (7) вывод в #login.

Используется SHA-256 (не HMAC), так как не нужен ключ. loginCounter гарантирует уникальность каждого нового логина.

5.4. Функция updateStrength() — индикатор надёжности

Критерии: длина (len) и количество типов символов (sets). Три уровня: strong (`len>=14` и `sets>=4`) — 3 сиреневых полосы, 'Отличный пароль', uwu; medium (`len>=10` и `sets>=3`) — 2 малиновых, 'Хороший, можно длиннее', :3; weak (остальное) — 1 розовая, 'Добавьте символов или увеличьте длину', >.<.

5.5. Функция `сру()` — копирование в буфер

Читает `#pwd`, копирует через `navigator.clipboard.writeText()`. При ошибке — запасной `execCommand('copy')`. Сообщение 'Скопировано!' на 1.5 сек.

5.6. Функция `saveCfg()` — сохранение настроек сервиса

Сохраняет `state`, `inv`, `len` для `curSvc` в `perSvc`. Использует `spread`-оператор для копирования объектов. Вызывается после `gen()` и `applyPreset()`.

5.7. Функция `getCfg()` — получение настроек сервиса

Если `perSvc[svc]` нет — создаёт со стандартом (`capitals=1`, `lowercase=1`, `digits=0`, `symbols=1`, `inv` пустые, `len=10`). Возвращает настройки.

5.8. Функция `applyCfg()` — применение настроек сервиса

Устанавливает `window.state`, `window.inv`, поле длины из `cfg`. Вызывает `renderChk()` и `renderInv()` для обновления интерфейса.

5.9. Функция `init()` — инициализация программы

Запускается при загрузке (последняя строка: `init()`). Вызывает `loadState()`, сбрасывает ошибки, устанавливает обработчики (`input` на `seed` — сброс ошибки, `keydown Enter` на `seed/service` — `gen()`, `blur` на `service` — смена сервиса). Применяет настройки для текущего сервиса. Вызывает `renderSvcChips()`.

5.10. Функция `loadState()` — очистка данных при запуске

Удаляет `'passgen_state'` из `localStorage` (`try/catch`). Сбрасывает `perSvc={}`, `svcs=[]`, `curSvc='(empty)'`, `state/inv` по умолчанию, `len=10`, очищает поле сервиса. Сессионный режим — данные не сохраняются между сессиями.

5.11. Функция onSvcChange() — смена сервиса

Читает значение из поля сервиса, обновляет curSvc, применяет настройки через applyCfg() если есть.

5.12. Функция saveState() — заглушка безопасности

Пустая. Ранее сохраняла в localStorage. После перехода на сессионный режим оставлена для совместимости.

5.13. Функция toggleInv() — переключение инверсий

Инвертирует window.inv[id] (0 ↔ 1). Обновляет класс кружочка (on/off). Вызывается при клике на опции 'Первая заглавная'/'Последняя цифра'.

5.14. Функция applyPreset() — применение пресетов

Пресеты: custom (без изменений), min (capitals+lowercase+digits, len=8), std (то же, len=10), max (все 4 типа, len=16). Вызывает renderChk() и saveCfg().

5.15. Функция exportData() — экспорт в JSON

Собирает настройки из perSvc для сервисов в svcs+'(empty)', формирует {svcs, perSvc}, JSON.stringify с отступами, Blob → URL.createObjectURL → скачивание 'passgen_export.json'.

5.16. Функция importData() — импорт из JSON

Читает файл через FileReader, парсит JSON, проверяет поля perSvc/svcs. Сохраняет в importDataCache, вызывает showImportModal(). Ошибки: 'Файл не содержит данных passgen', 'Ошибка импорта: неверный формат'.

5.17. Функция `showImportModal()` — окно выбора сервисов

Создаёт список сервисов из `d.perSvc` с чекбоксами (все отмечены). Если сервисов нет — 'Нет сервисов для импорта'. Открывает модальное окно.

5.18. Функция `confirmImport()` — подтверждение импорта

Собирает отмеченные чекбоксы, копирует `perSvc[svc]`=данные, добавляет в `svcs` если нет, сортирует, перерисовывает чипы. Применяет настройки для текущего сервиса, если он импортирован. 'Импорт завершен!' на 3 сек.

5.19. Функция `closeImportModal()` — закрытие окна импорта

Закрывает модальное окно, обнуляет `importDataCache`.

5.20. Функция `toggleAdvanced()` — расширенные настройки

Переключает `max-height` 0 ↔ 120px, `opacity` 0 ↔ 1. Стрелка поворачивается на 90 градусов.

5.21. Функция `renderInv()` — отрисовка инверсий

Обновляет классы кружочков `invCap/invDig` по `window.inv`. `true` → класс 'on' (закрашен), `false` → пустой.

5.22. Функция `renderChk()` — отрисовка переключателей

Создаёт 4 кружочка с подписями из `CHK`. Клик — инверсия `state[id]` и перерисовка. Вызывается при инициализации, смене сервиса, пресете.

5.23. Функция `adj()` — регулировка длины

Увеличивает/уменьшает длину на `d`. Границы: 1-99. Вызывается кнопками +/-.

5.24. Функция `toggleSvcPanel()` — панель сервисов

Переключает класс 'open' на `svcPanel` и `svcToggle`. Плавное появление/исчезание панели.

5.25. Функция `renderSvcChips()` — отрисовка чипов

Очищает `svcGrid`, отрисовывает все POPULAR (с логотипами), затем пользовательские из `svcs` (без логотипов, вопросительный знак), если их нет — 'Нет сохранённых сервисов'.

5.26. Функция `chip()` — создание чипа сервиса

Создаёт `div.svc-chip` с кружком (логотип или '?') и названием. Клик → подстановка в поле, закрытие панели, применение настроек. Если `deletable` — ПКМ удаляет сервис из `svcs`. Предустановленные сервисы (POPULAR) не удаляются.

6. Обработка ошибок

- Пустой сид — розовая подсветка с пульсацией, 'Введите сид'. Исчезает при вводе.
- Нет набора символов — 'Выберите хотя бы один тип символов'.
- Длина вне 1-99 — принудительная граница.
- Неверный JSON — 'Ошибка импорта: неверный формат файла'.
- Нет данных в JSON — 'Файл не содержит данных passgen'.
- Нет сервисов в импорте — 'Нет сервисов для импорта'.
- Не выбран сервис — 'Выберите хотя бы один сервис'.
- Clipboard недоступен — exesCommand как запасной.

7. Экспорт и импорт (формат JSON)

Пример структуры файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

- svcs — массив названий сервисов.
- perSvc — объект: ключ = сервис, значение = {state, inv, len}.
- state — флаги capitals, lowercase, digits, symbols (1/0).
- inv — флаги cap (первая заглавная), dig (последняя цифра).
- len — длина пароля.

8. Безопасность

- Локальность: всё в браузере, данные не уходят в сеть.
- Сессионный режим: данные очищаются при каждом запуске.
- Детерминированность: пароль восстанавливается из сида, не нужно хранить.
- Криптостойкость: HMAC-SHA256 с Rejection Sampling.
- Нет единой точки отказа: компрометация одного пароля не раскрывает другие.
- Устойчивость к length extension: HMAC вместо SHA-256.
- Невозможность подбора: без сида пароль не вычислить.

9. Запуск программы

Откройте index.html в любом современном браузере. Дважды кликните по файлу или перетащите в окно браузера. Программа не требует установки Python, Node.js или других компонентов. Всё необходимое уже есть в браузере.

Для Chromium-браузеров можно создать ярлык:

```
google-chrome --app=/путь/индекс.html --window-size=420,620
```

10. Схема вызова функций при генерации

10.1. Генерация пароля (нажатие 'Создать')

```
gen()
|-- Проверка seed (пусто -> ошибка, выход)
|-- Проверка state (все 0 -> ошибка, выход)
|-- Формирование alpha из СНК
|-- hmacGen(seed, svc, len, alpha)
|   |-- importKey('raw', seed, {HMAC, SHA-256})
|   |-- Цикл: msg=svc+len+ct -> sign(HMAC) -> 32 байта
|   |-- for byte: if byte<max: pwd+=alpha[byte%len(alpha)]
|-- Применение инверсий
|-- Вывод в #pwd, updateStrength(), копирование
|-- saveCfg() -> perSvc[curSvc]
```

10.2. Генерация логина (нажатие 'Создать логин')

```
genLogin()
|-- Очистка #login
|-- Проверка seed, проверка svc
|-- loginCounter++
|-- digest('SHA-256', seed+'login'+svc+counter)
|-- 6 байт -> alpha (без i,l,o,0,1) -> login
|-- Вывод в #login
```

11. Предустановленные популярные сервисы

1) Google — 'g' на синем (#4285f4). 2) GitHub — 'GH' на тёмном (#333). 3) VK — 'V' на синем (#4a76a8). 4) Yandex — 'Ya' на красном (#fc3f1d). 5) Mail — '@' на синем (#005ff9). 6) iCloud — 'i' на тёмно-синем (#369). 7) Onliner — 'O' на оранжевом. 8) Kufar — 'K' на зелёном.

12. Константы и структуры данных

12.1. Массив СНК — наборы символов

```
СНК = [
  ['capitals', 'Заглавные', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'],
  ['lowercase', 'Строчные', 'abcdefghijklmnopqrstuvwxyz'],
  ['digits', 'Цифры', '0123456789'],
  ['symbols', 'Спецсимволы', '!@#$%&*+=-.' ]
]
```

Из букв исключены I, L, i, l, o, O — их путают с цифрами. Из цифр исключены 0, 1.

12.2. Массив POPULAR — сервисы

```
POPULAR = [
  {n:'google', l:'g', c:'#4285f4'},
  {n:'github', l:'GH', c:'#333'},
  {n:'vk', l:'V', c:'#4a76a8'},
  {n:'yandex', l:'Ya', c:'#fc3f1d'},
  {n:'mail', l:'@', c:'#005ff9'},
  {n:'icloud', l:'i', c:'#369'},
  {n:'onliner', l:'O', c:'#ff6600'},
  {n:'kufar', l:'K', c:'#00a84d'}
]
```

12.3. Глобальные переменные

perSvc — объект настроек сервисов. curSvc — текущий сервис. svcs — массив пользовательских сервисов. loginCounter — счётчик (0,1,2...). window.state — флаги символов. window.inv — флаги инверсий. importDataCache — временные данные импорта.

13. CSS-переменные и стили

```
:root {
  --bg: #f5f3ff;      --card: #fff;      --fg: #1a1a2e;
  --border: #e5e7eb;  --muted: #999;      --shadow: rgba(90,20,140,.08);
  --pink: #f472b6;    --crimson: #e11d48; --lilac: #a855f7;
  --lilac2: #7c3aed;  --violet: #8b5cf6; --error: #f43f5e;
}
```

Анимации: fadeUp (появление), glow (пульсация ошибки), slideDown/slideUp (панели), checkPop (кружочки).

14. Полный список возможностей

- Генерация пароля на основе HMAC-SHA256.
- Генерация логина на основе SHA-256.
- Регулировка длины (1-99).
- Выбор типов символов (4 вида).
- Пресеты: Минимальный, Стандартный, Максимальный.
- Первая заглавная / Последняя цифра.
- Индикатор надежности (3 уровня).
- Автокопирование в буфер.
- Индивидуальные настройки для каждого сервиса.
- Панель популярных сервисов с логотипами.
- Автодобавление новых сервисов.
- Удаление сервисов по ПКМ.
- Экспорт/импорт JSON с выбором сервисов.
- Сессионный режим (очистка при запуске).
- Анимации и кастомные переключатели.
- Фиолетовая градиентная цветовая схема.

Пользователь
клик 'Создать'



gen()
запуск



Чтение DOM:
seed, svc, len,
state, inv

seed пуст?
-> ошибка



Розовая
подсветка
return



Типы символов
выбраны?



Ошибка
return



alpha = символы
из CHK по state

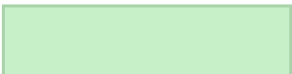


hmacGen(
seed, svc, len,
alpha)

seed = ключ HMAC
svc+len = сообщение

alpha = алфавит
max = 256-(256%len)

Цикл: msg = svc+len+счётчик -> importKey -> sign(HMAC) -> 32 байта -> byte<max? -> alpha[byte%len]



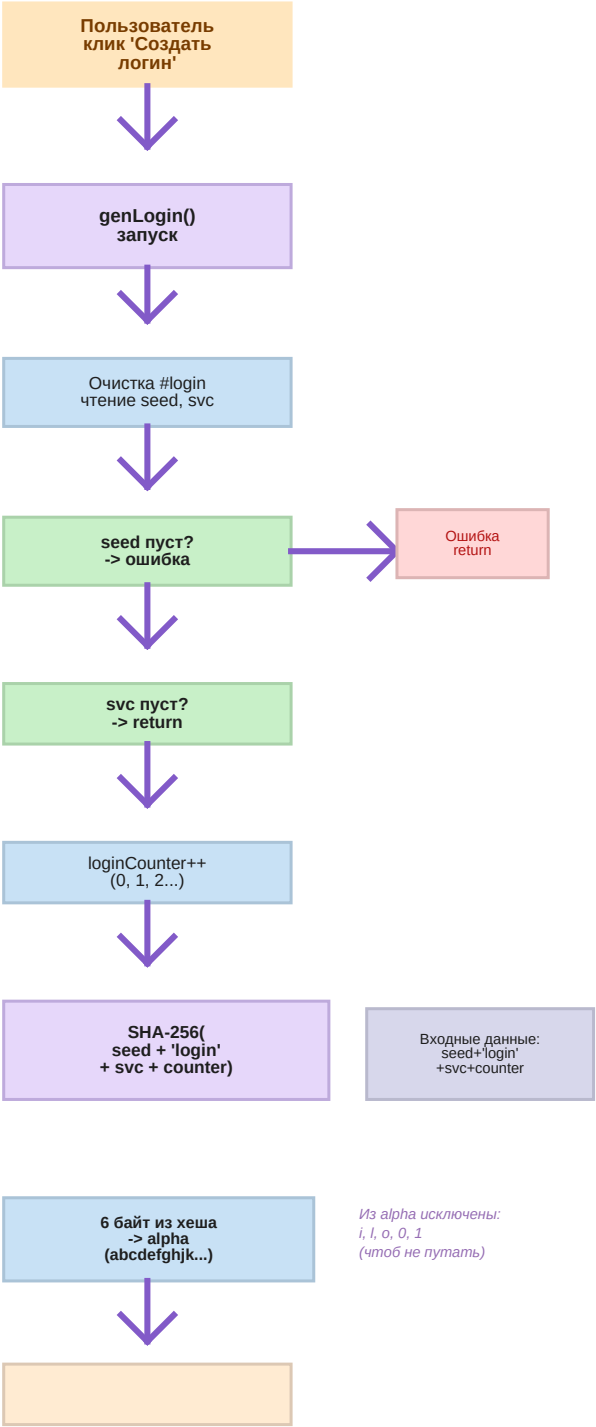
-> toUpperCase()

-> HMAC seed+

'_lastdig' -> цифра

updateStrength(len,sets)

saveCfg() -> perSvc



ГЕНЕРАЦИЯ ПАРОЛЯ

ГЕНЕРАЦИЯ ЛОГИНА

HMAC-SHA256: сид = ключ. svc+len+счётчик = сообщение. Rejection Sampling: $\max = 256 \cdot (256^{\text{len}(\alpha)})$. Байты $\geq \max$ отбрасываются.

SHA-256 (для логина): обычный хеш (не HMAC). loginCounter увеличивается при каждом клике, гарантируя уникальность.